

DIGITAL FORENSICS LABORATORY · LAB 01

Introduction to Digital Forensics

Working paper — Linux command-line foundations, evidence hashing, and file signatures

Name	Student ID (NIM)	Class / Group
------	------------------	---------------

Date	Instructor / TA	Session grade
------	-----------------	---------------

1. Lab Overview

This working paper accompanies Lab 01 of the digital forensics series. It is yours to fill in: complete the pre-lab check before the session, check off each step as you perform it, answer every analysis question with numbered evidence, and close with a reflection. The companion slide deck covers the same theory in lecture form; this document is where the actual work is recorded.

1.1 Learning objectives

- Explain what digital forensics is, the three stages of the process, and where it is applied.
- Operate the twelve core Linux commands used throughout the whole lab series.
- Use MD5 and SHA1 hashing to prove evidence integrity, and explain the avalanche effect.
- Identify file types by magic bytes rather than by extension.
- Repair a damaged file header with a hex editor on a working copy.
- Hunt evidence in a multi-thousand-line access log using grep.
- Document findings with numbered exhibits so every claim is traceable.

1.2 Environment and files

Environment: Kali Linux (VM or live USB) with internet access for one package installation. All exercise files are inside the Lab 01 folder of the course repository. Copy them into your own working directory first — never analyze the originals in place.

Table 1.1 — Lab files

File	Purpose
files/challenge.png	Challenge A — corrupted PNG from the original lab
files/challenge_extra.png	Challenge B (new) — a file that lies about its type

File	Purpose
files/server.log	Small auth log for the guided search drill (Section 3.C)
files/access_big.log	Challenge C — web access log, 8,600+ lines
files/update.bin	Challenge C — suspicious binary artifact for strings
files/evidence_A.txt / evidence_B.txt	Hash comparison pair for Section 3.D
files/not_a_jpeg.jpg	Extension-disguise sample for Section 3.E
files/images/	Reference images used by the lab

1.3 What to submit

- This working paper as PDF, named NIM_Name_Lab01.pdf.
- Your evidence folder NIM_Lab01/evidence/ containing every numbered exhibit (B-01, B-02, and so on).
- The two recovered challenge images (working copies only — originals must remain untouched).

2. Theory Primer

2.1 Digital forensics at a glance

Digital forensics is the process of using technology to gather evidence, investigate it, and present the findings in a legal case. The scope ranges from network activity and access logs to search history and storage media such as hard disks and mobile devices. The work follows three stages — collect, analyze, present — and the common goal is evidence strong enough to be used in a court of law.

Table 2.1 — Where digital forensics is applied

Use case	What the investigator does	Example scenario
Investigating cyber attacks	Determine the scope and source of a breach	A web server is defaced; logs reveal which exploit was used and from where
Threat detection & response	Proactively identify and mitigate threats	An analyst spots repeated failed logins and blocks the source before damage occurs
Data recovery	Recover stolen or deleted data	Documents deleted during an intrusion are recovered from a disk image
Criminal investigations	Identify suspects and establish motive	Browser history and file timestamps link a suspect to a fraud scheme

2.2 The digital trail

Every digital action leaves evidence: file access timestamps, network connection logs, process execution records, browser history, and deleted file remnants. Finding and correctly interpreting this trail is the investigator's core job. Keep this in mind during every lab in the series — the artifacts change (registry, memory, packets), the principle does not.

2.3 The command toolbox

Twelve commands carry an entire first investigation. You will keep using all of them in later labs.

Table 2.2 — Core commands

Command	What it does	Forensic use
<code>ls [-la]</code>	List directory contents	Inventory a folder, including hidden dotfiles
<code>cd</code>	Change working directory	Move between evidence locations
<code>cat</code>	Print a file's contents	Read notes, configs, small logs
<code>strings [-n N]</code>	Extract printable sequences from any file	Surface URLs, keys, and messages hidden in binaries
<code>grep [-i] [-c]</code>	Search a pattern in files	Filter huge logs down to relevant lines
<code>find</code>	Locate files by criteria	Find recently modified or suspicious files
<code>file</code>	Identify a file by its content	Detect files disguised by a false extension
<code>md5sum / sha1sum</code>	Compute a hash checksum	Prove evidence has not been altered
<code>xxd</code>	Hex dump of a file	Inspect headers and raw bytes
<code>hexedit</code>	Edit raw bytes interactively	Repair corrupted headers on a copy
<code>ps</code>	List running processes	Spot unfamiliar or masquerading processes
<code>netstat</code>	Show network connections	Map active connections and their endpoints

2.4 Integrity and hashing

A hash function converts any input into a fixed-length string called a checksum. Identical content always produces an identical hash; change a single bit and the hash changes completely (the avalanche effect). That property makes hashing the standard proof that evidence was not tampered with between acquisition and reporting. The professional rules are simple:

- Hash the evidence the moment you receive or create it, and record the value.
- Work only on copies — the original stays sealed.
- Re-hash after analysis and compare: identical values prove your copy is faithful.

Worked example — compute and compare:

```
$ md5sum evidence_A.txt
a1b2c3d4e5f6a7b8c9d0e1f2a3b4c5d6  evidence_A.txt
```

```
$ sha1sum evidence_A.txt
9f8e7d6c5b4a39281706f5e4d3c2b1a098765432  evidence_A.txt
```

2.5 File signatures (magic bytes)

Every file format begins with a fixed byte sequence — its signature, or magic bytes. The file command reads this signature instead of trusting the extension, which is how investigators catch disguised files. When a header is damaged, the data may still be intact: restoring the correct magic bytes on a working copy often recovers the file.

Table 2.3 — Common file signatures

Format	First bytes (hex)	Notes
JPEG	FF D8 FF E0	Photos; EXIF metadata follows
PNG	89 50 4E 47 0D 0A 1A 0A	8-byte signature; IHDR chunk at offset 12
PDF	25 50 44 46 2D	Reads as %PDF-
BMP	42 4D	Reads as BM
ZIP	50 4B 03 04	Also the prefix of .docx and .xlsx files
ELF	7F 45 4C 46	Linux executables

3. Guided Exercises

Work through the five drills in order. Every task that produces output is an exhibit: capture it, number it (B-01, B-02, and so on), and log it in Appendix B. Commands shown in gray boxes are examples — your outputs will differ, and that is expected.

3.A Navigation warm-up (ls, cd, find)

- **1.** Create your working directory ~/forensics/lab01 and copy every lab file from Table 1.1 into it. This copy is what you analyze all session.
- **2.** List the contents of your working directory, then of your home directory, including hidden files. Note at least two entries that plain ls did not show.
- **3.** Count the JPG and PNG files under files/images. Record the exact command used.
- **4.** Find every file inside your home directory modified in the last 30 minutes.

3.B Reading content (cat, strings, file)

- **1.** Print evidence_A.txt with cat and read its contents.
- **2.** Run strings on /bin/bash and compare the output with cat on the same file. Explain the difference in one sentence in your evidence notes.
- **3.** Extract printable strings of minimum length 8 from /bin/bash. Record which option you used.

- 4. Run file on every file you copied in 3.A. Which files does file identify differently from what their extension suggests? (not_a_jpeg.jpg counts — explain why.)

3.C Pattern hunting (grep) — case file: server.log

server.log in files/ records one morning of authentication activity on a small server. Something happened between 08:14 and 08:23, and there is more noise later in the morning. Use grep (with find where useful) to answer the drills below — they rehearse exactly the skills that Challenge C and the later web-forensics lab demand. Treat this small log as the guided rehearsal: its big sibling access_big.log is your solo performance in Section 4.3.

- 1. Count all failed password attempts in the log.
- 2. List every unique source IP that produced a failed attempt, with its count. One address clearly dominates — which?
- 3. Did the attacker eventually get in? Find the successful login from the attacking IP and note the exact time and account name.
- 4. Find the privilege-escalation attempt involving /etc/shadow. Which account tried it, and did it succeed?
- 5. Identify the second, smaller brute-force attempt from a different IP around 08:44.
- 6. Determine when the attacking IP was finally disconnected, and why (find the log line).

Useful patterns for this drill:

```
grep "Failed password" server.log
grep "Failed password" server.log | grep -c "203.0.113.77"
grep "Accepted password" server.log
```

3.D Hashing drills (md5sum, sha1sum)

- 1. Compute the MD5 and SHA1 hashes of evidence_A.txt and evidence_B.txt. Compare the pairs and state whether the files are identical.
- 2. Open both files with cat. Can you see any difference? Check the file sizes with ls -l and explain what is going on.
- 3. Create your own file, hash it, then modify one character and hash it again. Paste both hashes and describe the avalanche effect you observe.
- 4. Hash challenge.png and challenge_extra.png now, before the challenges — these are your acquisition hashes. Record them in Appendix C.

3.E File types and hex (file, xxd)

- 1. Run file on not_a_jpeg.jpg. What does it report, and what does that tell you about its true format?
- 2. Dump the first line of not_a_jpeg.jpg with xxd and match the bytes against Table 2.3.

- **3.** Dump the first 16 bytes of `challenge_extra.png` with `xxd`. The output claims a certain format — which one?
- **4.** Look further into `challenge_extra.png` (bytes around offset 12). Which chunk name appears there, and which format does it actually belong to? Write down your conclusion.

4. Challenges

All challenges hide flags with the format `ctf{xxxx}`. Rules: work on copies, hash before and after, and record every command as an exhibit. Hints are staged — try to stop after each one and attempt the challenge again before reading further.

4.1 Challenge A — `challenge.png` (original lab)

- **1.** Copy `challenge.png` to `challenge_A_work.png` and work only on the copy.
- **2.** Confirm the damage: try opening it, then inspect the header with `xxd`.
- **3.** Compare the header bytes against the true PNG signature from Table 2.3.
- **4.** Repair the signature with `hexedit` and save.
- **5.** Open the repaired image and record the flag.
- **6.** Re-hash both the original and your repaired copy; explain why the hashes differ.

Hints: (1) file says the type is merely data — the header is wrong. (2) Only the first bytes are damaged. (3) hexedit: navigate with arrows, type hex values, F2 saves, Ctrl+C exits.

Flag A: `ctf{_____}`

4.2 Challenge B — `challenge_extra.png` (new)

- **1.** Copy `challenge_extra.png` to `challenge_B_work.png`.
- **2.** Run `file` on it. It confidently claims a type — do you believe it?
- **3.** Inspect the first 16 bytes with `xxd`. Then look at offset 12 and beyond.
- **4.** Decide which signature the file should really carry, and repair all wrong bytes.
- **5.** Open the repaired image and record the flag.

Hints: (1) Two different formats share this story — one in the header, one in the chunks. (2) The IHDR chunk at offset 12 only exists in one format family. (3) The first four bytes are a lie; so are the next four. Restore all eight.

Flag B: `ctf{_____}`

4.3 Challenge C — log hunt: `access_big.log` + `update.bin`

A web server's access log from 26 September 2026 sits in `files/access_big.log` — 8,600+ lines of ordinary shopping-site traffic with one intrusion story buried in it. The attacker also left a binary

artifact, `update.bin`, in the same folder. Your mission: reconstruct the timeline and recover two hidden secrets.

- **1.** Copy `access_big.log` and `update.bin` to your working directory (never touch the originals).
- **2.** Count how many requests each client IP generated, and identify the single most active external address. Record the command you used.
- **3.** List the requests from that address that returned 404. What kind of paths are being probed, and what does that tell you?
- **4.** Identify at least two automated scanning tools by their User-Agent strings, and count the lines each appears on.
- **5.** Reconstruct the attack timeline: brute-force window on `login.php`, the first successful login, the upload, and the follow-up fetches — note timestamps for each stage.
- **6.** Somewhere the attacker leaked a secret, URL-encoded, inside a request. Find the line, then decode the value to recover the first flag.
- **7.** Run `strings` on `update.bin` (with a sensible minimum length) and hunt for configuration-style entries: a URL, a key, a service name — and the second flag.
- **8.** Write a five-line incident summary: who, when, what was probed, what succeeded, and what was left behind.

Hints: (1) Sorting by count is one pipeline away: `grep`, then counting per IP. (2) The scanners announce themselves politely in the User-Agent field. (3) URL encoding turns braces into `%7B` and `%7D` — `grep` for what remains visible, then decode. (4) In `update.bin`, plain strings is enough; a minimum length of 6 keeps it tidy.

Flag C1 (from the log): `ctf{_____}`

Flag C2 (from `update.bin`): `ctf{_____}`

5. Analysis Questions

Questions 1-9 come from the original lab; questions 10-12 are new scenario questions. For command questions, write the exact command; for conceptual questions, answer descriptively. Every factual claim needs an exhibit number.

1. Which command lists all `.jpg` files in the current directory?

Answer:

Write your complete answer, including the exact commands used.

Supporting evidence (screenshot / output): exhibit numbers B-____ to B-____

2. Which command reads the contents of /etc/passwd?

Answer:

Write your complete answer, including the exact commands used.

Supporting evidence (screenshot / output): exhibit numbers B-____ to B-____

3. Which command searches for the string Error in all files under /var/log?

Answer:

Write your complete answer, including the exact commands used.

Supporting evidence (screenshot / output): exhibit numbers B-____ to B-____

4. Write the commands that compute the MD5 and SHA1 hashes of /etc/passwd.

Answer:

Write your complete answer, including the exact commands used.

Supporting evidence (screenshot / output): exhibit numbers B-____ to B-____

5. Which command prints all printable strings of length 8 or more from /bin/bash?

Answer:

Write your complete answer, including the exact commands used.

Supporting evidence (screenshot / output): exhibit numbers B-____ to B-____

6. The file command reports: image.jpg: ELF 64-bit LSB pie executable, x86-64. What is wrong with this file, and what is the forensic significance of your conclusion?

Answer:

Write your complete answer, including the exact commands used.

Supporting evidence (screenshot / output): exhibit numbers B-____ to B-____

7. Which command finds files modified within the last 30 minutes under /home?

Answer:

Write your complete answer, including the exact commands used.

Supporting evidence (screenshot / output): exhibit numbers B-____ to B-____

8. Which command displays information about all active TCP connections?

Answer:

Write your complete answer, including the exact commands used.

Supporting evidence (screenshot / output): exhibit numbers B-____ to B-____

9. Reconstruct your Challenge A recovery: magic bytes found, repair applied, flag, and hashes before and after. Write it as a short evidence narrative.

Answer:

Write your complete answer, including the exact commands used.

Supporting evidence (screenshot / output): exhibit numbers B-____ to B-____

10. An investigator receives two files whose SHA1 hashes match. What can be concluded, and what cannot? Explain in 3-4 sentences.

Answer:

Write your complete answer, including the exact commands used.

Supporting evidence (screenshot / output): exhibit numbers B-____ to B-____

11. A suspect renamed a malware executable to vacation.jpg to smuggle it past a quick glance. Which single command reveals the disguise, what output would you expect, and which signature gives it away?

Answer:

Write your complete answer, including the exact commands used.

Supporting evidence (screenshot / output): exhibit numbers B-_____ to B-_____

12. Why must forensic analysis be performed on copies rather than originals? Describe the chain of custody in three steps, from seizure to report.

Answer:

Write your complete answer, including the exact commands used.

Supporting evidence (screenshot / output): exhibit numbers B-_____ to B-_____

6. Reflection and Self-Assessment

Summarize your main findings, the skills that clicked, and the obstacles you hit — with how you solved them.

Reflection:

Findings, new skills, obstacles, and how you overcame them.

Supporting evidence (screenshot / output): exhibit numbers B-_____ to B-_____

Table 6.1 — Self-assessment checklist

I can	Yes	Not yet
Explain digital forensics and its three stages		
Use all twelve core commands without looking them up		
Prove file integrity with MD5/SHA1 and explain the avalanche effect		
Identify a file's true type from its magic bytes		
Repair a damaged PNG header on a working copy		

I can	Yes	Not yet
Document findings as numbered, traceable exhibits		

7. Appendices

Appendix A — Command cheat sheet

Task	Command
List files (all, long)	<code>ls -la</code>
Read a text file	<code>cat file.txt</code>
Extract strings (min length 8)	<code>strings -n 8 file.bin</code>
Search a pattern (ignore case, count)	<code>grep -i pattern file</code> / <code>grep -c pattern file</code>
Find by name / by modification time	<code>find / -name "*.jpg"</code> / <code>find /home -mmin -30</code>
Identify true file type	<code>file suspect.jpg</code>
Hash (MD5 / SHA1)	<code>md5sum evidence.dd</code> / <code>sha1sum evidence.dd</code>
Hex dump (first line)	<code>xxd file.png head -1</code>
Edit bytes	<code>hexedit copy.png</code>
Processes / connections	<code>ps aux</code> / <code>netstat -tn</code>

Appendix B — Evidence log

One row per exhibit. Keep numbering continuous across the whole session.

No.	Time	Activity / Command	Source file / Tool	Notes
B-01				
B-02				
B-03				
B-04				
B-05				
B-06				
B-07				

No.	Time	Activity / Command	Source file / Tool	Notes
B-08				
B-09				
B-10				

Appendix C — Hash verification form

Fill one row per evidence file: hash when received (start) and after analysis (end). Identical values prove faithful handling.

No	Evidence file	Algorithm	Hash (start)	Hash (end)	Match?
1					Yes / No
2					Yes / No
3					Yes / No
4					Yes / No
5					Yes / No
6					Yes / No

Honesty statement: by submitting this working paper I declare that all answers and evidence are my own work, that original evidence files were never modified, and that I followed the lab's ethics rules.

Student signature: _____ Date: _____

Instructor / TA signature: _____ Grade: _____